Rewrite — ein Mathematiksystem

Rainer Heintzmann*

3. November 2018

Zusammenfassung

Rewrite ist ein Sytem zur Beieinflußung symbolischer Ausdrücke, und zum Durchführen numerischer Rechnungen. Mittels Umschreiben der Ausdrücke in einem Regelsystem von 'Algebren' lassen sich einfache algebraische Funktionen wie Differenzieren, Lösen von quadratischen Gleichungen usw. von Termen ausführen. Die Regelsysteme sind selbst zu definieren. Unter dem Verzeichnis 'inc' sind einige Beispiele.

Numerisches Auswerten der Terme ist auch möglich. Benutzer können eigene Funktionen schreiben und sie in die Liste vorhandener Funktionen in dem Modul 'userfkt.c' einfügen.

1 Starten des Programms

Starten sie das Programm 'rewrite' von der shell aus. Geben Sie nun 'INC "inc/all"' ein. Es werden nun eine Reihe von Include-Files geladen. Nun können Sie Ihre Aufgabe stellen z.B: 'REWR D(sin cos x,x) : All' Das Programm bestimmt nun die Ableitung der Funktion 'sin(cos(x))' nach x durch Umschreiben dieses Terms in der Algebraliste 'All'.

2 Benutzung von 'Rewrite'

Generell wird die Eingabe immer Zeilenweise verarbeitet. Bei größeren Blöcken muß die Zeile durch ein '\' am Zeilenende fortgesetzt werden. Wichtig ist, daß kein Zeichen (auch kein Leerzeichen) hinter dem '\' steht. Kommentare beginnen mit '#' und gehen bis zum Ende der Zeile oder bis zum nächsten '\' gefolgt von einem Zeilentrenner.

*Blumenhaller Weg 138,49078 Osnabrück

3 Liste aller Rewritebefehle

Befehl Erklärung

INC "filename" Unter 'filename' abgespeicherte Programme werden ausgeführt. 'Filename' ist ein von '"' umschlossener String. Mehrfache INCLUDES sind möglich bis zu einer maximalen Tiefe.

Bsp.: INC ''inc/all" : alle wichtigen Include-files werden gelesen.

Der INC-Befehl wird einmalig beim verarbeiten der Eingabe ausgeführt. Zeilen, die 'INC' enthalten dürfen nicht mit '\' fortgesetzt werden. (Einzige Ausnahme !)

REWR term : algname [;algname ..] Der Term 'term' wird in den Algebren (Regellisten) 'algname' nacheinander umgeschrieben.

PRTNUM "String" Das Standartformat für die Ausgabe von reelen Zahlen wird in 'String' geändert. Z.B. : PRTNUM "% 2.15g "

OUT "filename" Die Ausgabe wird in das File 'filename' umgelenkt; dieses wird vorher gelöscht. Zusätzlich ist noch 'OUT "stdout"' möglich. Dadurch findet die Ausgabe wieder auf stdout (normalerweise der Bildschirm) statt. Bei jeder Ausgabeänderung wird die noch offene Datei geschlossen. Grafische Darstellung von Funktionen mittels anderer Programme ist so einfach möglich.

DATA Const Liest eine Real-Zahl aus der geöffneten Datei in die Konstante 'Const'. So lassen sich z.B. auch Meßwerte weiterverarbeiten.

DATA "filename" Zusätzlich zu der normalen Standarteingabe besteht die Möglichkeit mit 'DATA Const' aus einer Datei zu lesen. Diese Datei muß mit 'DATA "filename"' geöffnet werden. Während des Lesens wird beim Erreichen von nicht numerischen Daten der Rest, bis zum Zeilenende übersprungen.

SYS "String" fürt den in 'String' angegebenen Befehl in der shell aus.

SORT name [,name ..] Erzeugung eines Datentyps. Der Typ wird bei der Definition von Funktionen, Variablen und Konstanten gebraucht.

NUM sortname Standartmäßig wird mit dem Datentyp 'double' gerechnet Alle eingegebenen Zahlen (z.B.: 1.234) werden als 'double' abgespeichert. Sie müssen aber einem selbstdefiniertem Datentyp (mit 'SORT typname' erzeugt) zugeordnet werden.

Bsp.: SORT real

NUM real # ordnet Nummern den Typ real zu.

CONST name [,name ..] : sortname Deklaration einer Konstanten. Es wird eigentlich eine Konstante erzeugt, die beliebige Werte annehmen kann. (Also eine Variable). Im Gegensatz zu mit VAR erzeugten Metha-Variablen kann sie aber nicht als Ersatz für ganze Terme benutzt werden.

VAR name [,name ..] : sortname Erzeugung einer Metha-Variablen. Man kann diese Variable als Platzhalter in Spezifikationen für Algebren (Regellisten) verwenden.

Bsp.: VAR A : real SPEC xyz A+0=A

NUM name [,name ..] : sortname Erzeugung einer Metha-Variablen. Man kann diese Variable als Platzhalter nur für Numerische Werte in Spezifikationen für Algebren (Regellisten) verwenden.

Bsp.: NUM N : real

SPEC xyz N+0=N

FUNC name : sortlist >> **sortname [LA1..5—RU] [ATT name]** Erzeugung einer Funktion. Die erste 'sortlist' gibt die Anzahl und Typen der Argumente an. Es muß mindestens eines sein. Sie werden jeweils durch Kommata voneinander getrennt. Ein ++ hinter dem letzten Argument steht für variabel viele Argumente des letzen Types. Der andere 'sortname' gibt den Resulttattyp der Funktion an. LA1 bis LA5 legt das Ein- bzw. Ausgabeformat der zu definierenden Funktion fest. Es bedeuten:
LA1 bis LA5 : Links-assoziativer Operator (2 Argumente) LA5 klammert vor LA4 usw.
RU : Rechts-unärer Operotor. (z.B. Fakultät)
RA : Rechts-assoziativer Operator (z.B. $a\hat{b}$)

Ist kein spezielles Ein/Ausgabeformat angegeben, so wird die Funktion in der Form : 'sin x' , 'sin(x)' bzw. 'f(x,y)' eingelesen.

ATT name legt den Namen der Funktion in der Liste 'userfunction' im Modul userfkt.c fest, die aufgerufen wird, wenn alle Argumente der Funktion einen numerischen Wert besitzen. 'name' ist nicht in '"' zu setzen. Name darf frei gewählt werden (auch gleichlautend mit einem Funktionsnamen).

z.B.: FUNC + : real,real >> real LA1 ATT 2

auch möglich ist:
ATT (Argdefs) Block Hängt eine selbstdefinierte Funktion an. (Argdefs) sind vollständige Definitionen für die Argumente (Konstanten,Terme oder Funktionen). Der entsprechende Funktionskörper wird ausgeführt, wenn eine Zuweisung eines Terms an eine Konstante erfolgt, oder ein Term ausgegeben werden soll.

z.B. FUNC test : real,real,real >> real ATT \
(CONST a,b : real; TERM c : real) { RETURN c+a*b }

RETURN Term Setzt den Resultatwert (einen Term) der Funktion. Die Ausführung der Funktion wird NICHT!! beendet. Enthält dieser Term lokale Variablen, so gibt es eine Fehlermeldung, und die Funktion gild als nicht ausgeführt.

OTHER name : sortlist >> sortname [ATT name] Definiert zu einem existierenden Funktionsnamen eine weitere Abbildung mit anderen Argument- und Resultattypen. Funktionen können anhand ihrer Argumenttypen unterschieden werden.

z.B.: OTHER + : vector,vector >> vector

BRA auf zu : sortlist >> sortname [ATT name] Definiert eine Funktion, die in Klammerschreibweise benutzt werden kann. 'auf' bezeichnet das Symbol für Klammer auf, und 'zu' das Symbol für Klammer zu. Es ist folgende zusaetzliche Notation bei der Benutzung erlaubt:

a[b,c] statt [a,b,c] ([= auf,] = zu) damit lassen sich z.B. Vectoren mit a[5] indizieren.

z.B: BRA <>: vector,vector >> real # Skalarprodukt <a ,b >
Kennzeichnend für OTHER ist die erste Klammer.
z.B.: OTHER <: compl,compl >> real

ALG algname Definiert eine neue Algebra (Regelliste) , die Spezifikationen enthalten kann

SPEC algname : [Fkt ...] was = zuwas [DO Block] Alte Spezifikationen werden gelöscht, diese neue Regel wird eingetragen.

SPEC algname : algnamelist Alte Spezifikationen werden gelöscht, 'algname' kann anstelle der gesamten algnamelist verwendet werden. In der 'algnamelist' stehen 'algnames', die durch Kommata voneinander getrennt sind. Hintereinander Umschreiben in mehreren Algebren kann so elegant geschrieben werden. Prinzipiell dürfen Listen von Algebren immer mit '(' ')' Klammern umgeben werden, was bedeutet, daß beim Umschreiben die Algebren innerhalb der Klammern solange ausgeführt werden, bis kein weiteres Umschreiben mehr möglich ist.

ADDSPEC algname : [IF (Fkt)] was = zuwas [DO Block] Hinzufügen einer Ersetzungsregel (Spezifikation) zur Algebra 'algname'. Es dürfen Metha-Variablen auf beiden Seiten verwendet werden. Mit NUM definierte Metha-Variablen werden nur bei Zahlen ersetzt.

Fkt ist selbstdefiniert (siehe IF) oder eine der Funktionen :

LB(term,term) 'Lower or between' vergleicht zwei Terme.
ISINT(term) stellt fest ob term ein int-wert ist.
IN(terma,termb) stellt fest ob terma in termb enthalten ist.
NIN(terma,termb) verneinung von IN.

Abhängig vom Resultat der Funktionen wird die Regel angewandt oder eben nicht. Das Hintereinanderschreiben mehrerer Funktionen bewirkt eine und-Verknüpfung der Resultate.

Bsp.: ADDSPEC real : LB(A,B) A+B=B+A
Ist DO in Verknüpfung mit einem Block (oder einem Befehl) angegeben, so wird vor dem Ersetzen der linken durch die rechten Seite der Block ausgeführt. Mehrere Befehle müssen in dem Block von '{' und '}' umgeben sein.

Bsp.: ADDSPEC real : Tay(A,B,D)=q DO { q=0
FOR i=1 TO N STEP 1 DO \
q=q+Dî/i!*Dp(B,D) }

LIFT (Fktlist(Args)) TO (Fktlist(Args)) Übertragung sämtlicher auf eine Menge von Funktionen und Konstanten zutreffender Regeln auf eine andere solche Menge. Fktlist(Args) ist eine durch Kommas getrennte Liste von Funktionen mit irgentwelchen Argumenten des gewünschten Datentyps oder Konstanten.

Die jeweiligen Funktionen werden durch den Typ ihrer Argumente eindeutig charakterisiert. Die Argumente spielen sonst im weiteren keine Rolle. Als Konstanten sind auch die reellen Zahlen 1 und 0 möglich.

Alle Regeln, die in der linken Menge gelten werden nun automatisch auch auf die rechte Menge angewandt.

Bsp. :

```
LIFT (Aag+Aag,Aag-Aag,-Aag,0ag) TO (Ar+Ar,Ar-Ar,-Ar,0r)
LIFT (Aag+Aag,Aag-Aag,-Aag,0ag) TO (Ar*Ar,Ar/Ar,mi(Ar),1r)
```

mit diesem Trick lassen sich die Eigenschaften eines Ringes leicht aufgrund schon spezifizierter Gruppeneigenschaften definieren.

TERM name : sortname Deklaration einer Term-Variable. Eine Term-Variable kann einen beliebigen Ausdruck des gleichen Typs aufnehmen.Ineianderschachtelung von Term-Variablen bewirkt ein Einsetzen der aktuellen Ausdrücke. Die Abhängigkeit zu der anderen Term-Variablen wird nicht gespeichert.

Immer vorhanden ist der Term '%'. Er enthält den jeweils zuletzt ausgegebenen Ausdruck. Soll dieser weiterhin gespeichert werden, so muß er an eine andere Term-Variable zugewiesen werden.

Term-Variablen werden eigentlich im Sinne von Funktionen verwendet. So ist die Zuweisung 'TERM q q=xŷ*a' im Sinne von 'q(x,y,a)' zu verstehen. Durch Zuweisungen an die einzelnen Variablen 'x=4 y=2 a=3' kann dann einfach der Funktionswert von q(4,2,3) mit 'q' ausgerechnet werden.

ATT sortname Name1 , Name2 , Name3 , Name4 Einem Typ wird Anzeige-,Vergleichs- , Kopier- und Löschfunktion für numerische Werte zugewiesen. Dei 'Namen' stehen hierbei nicht in doppelten Anführunsstrichen.

'Name1' bezeichnet die Anzeigefunktion in 'userfunction' mit der numerische Werte vom Typ sortname angezeigt werden sollen.
'Name2' bezeichnet die Vergleichsfunktion mit der zwei numerische Werte verglichen werden (1 : Zahl1 > Zahl2,0: Zahl1 == Zahl2, -1 : Zahl1 <Zahl2).
'Name3' bezeichnet die Kopierfunktion in 'userfunction' mit der Numerische Werte kopiert werden.
'Name4' bezeichnet die Löschfunktion in 'userfunction' mit der Numerische Werte gelöscht werden.

Bsp.: ATT complex show_complex,cmp_complex,cpy_complex,del_complex
siehe Modul userfkt.c

FOR Constname = N1 TO N2 [STEP N3] DO Block For-Schleife.

Bsp.: FOR i = 20 TO 10 STEP -1 DO i

IF (Fkt) Block [ELSE Block] IF-Bedingung. Fkt ist der Name einer Funktion mit double Resultat, die TRUE/FALSE (1.0/0.0) zurückgibt.

Bsp.: CONST inc_xxx : real
inc_xxx = 1
IF (inc_xxx >0) INC "inc/xxx"

WHILE (Fkt) Block WHILE-Schleife. Für 'Fkt' siehe 'IF'.

Bsp.: i = 1
WHILE (10>i) 2^i

Zuweisungen: Termname = term term wird in Termname gespeichert

Bsp.: TERM q : real
q=1+1+a

Der letzte ausgegebene Ausdruck ist unter '%' als Termname immer verfügbar. Er wird also automatisch an '%' zugewiesen. In selbstdefinierten Funktionen dürfen lokale Variablen nicht an globalere Terme zugewiesen werden.

(daraus resultiert eine Fehlermeldung bei der Ausgabe. (Zuweisung an %)) Werden Term-Variablen an Term-Variablen zugewiesen, so wird der Inhalt kopiert und gegebenenfalls numerisch ausgewertet. Wird also eine Term-Variable verändert hat dies keinen einfluß auf andere Term-Variablen.

Constname = value Die Konstante 'Constname' bekommt einen Wert

z.B.: x=1+1+3 (wenn + mit Funktion verbunden)

Steht bei 'value' kein Wert (z.B. andere Konstante ohne Wert oder Funktion, die nicht rechnet) so wird der Wert der Konstanten gelöscht und sie erscheint in der Ausgabe wieder mit ihrem Namen 'Constname'.

Sowohl eine Const- als auch eine Term-Variable können durch einfaches hinschreiben des Namens ausgegeben werden. Ausgegebene Ausdrücke werden an '%' zugewiesen.

zur Verdeutlichung ein Beispiel :

```
1+1
2                   # wird ausgegeben
TERM q : real
q=1+1
q
2                   # wird ausgegeben (q bleibt aber erhalten)
Der letzte Ausgedruckte Wert ist unter dem Term \% wieder zu erreichen.
Bestehende Abh"angigkeiten zu anderen Termen werden hier ignoriert.
Bsp.:
TERM q
q=b
a+q
a+b                 # wird ausgegeben
q=c
%+2
a+b+2               # wird ausgegeben
```

4 Allgemeines

Werden von '"' begrenzte Strings hingeschrieben, so wird der Text ausgegeben. Sie können nicht zugewiesen werden. (ein String darf auch '"' enthalten.)

Eine Reihe 'benutzerdefinierter' Funktionen sind schon implementiert. Dazu gehören wichtige Funktionen auf reelen Zahlen (real.c), complexen Zahlen (complex.c), reelen Vectoren (rvector.c) und reelen Matrizen (rmatrix.c) und Verbindungen der Datentypen. Alle Funktionen sind im Modul (userfkt.c) eingetragen.

Neue Funktionen müssen immer ein array von Zeigern auf Argumente als Argument bekommen und einen Zeiger auf das Resultat zurückliefern. Beispiele sind in den genannten c-Modulen zu finden.

Die grundlegenden mathematischen Datentypen, Funktionen und Regeln sind im Modul 'alg' im Verzeichniß 'inc' definiert. Dort kann man auch nachlesen welche mathematischen Funktionen definiert sind.

Wichtigste Fehler des Systems:
Der Speicher wird nur unzureichend freigegeben.
Mängel des Systems:
Wünschend währe ein mehr Objekt-orientierter Ansatz; einfache Dinge wie differenzieren usw. sind dann deutlich schneller (verschicken von Nachrichten). Es existieren keine bisher 'Container-Datentypen' wie Arrays, Matritzen, usw. . Diese müssen durch entsprechende Funktionen nachgebildet werden, was Rechenzeit kostet.

A Zusammenfassung der yacc-syntax

```
%token CST VAR SORT FUNC ALG SPEC ADDSPEC REWR PRT GRAPH PFEIL TERM NUM ATT IS EQ
%token LA1 LA2 LA3 LA4 LA5 LOCAL RU RA KOMMU LB IN NIN PARAM FOR TO STEP DO DBG LAST
%token BRA OTHER DATA OUT INC EXE VARPAR SYS IF ELSE WHILE PRTNUM LIFT QUIT RETURN
%token <y\_sym> Id String
%token <y\_sym> Sort Alg Term
%token <y\_sym> La1 La2 La3 La4 La5 Ru Ra Func Const Var Bra Bra1 Bra2
%token Terms Form Sorts Functs Forms Algs Cond2 Cond
%type <y\_int> VORRANG
%type <y\_nod> vinstr definition instruction aninstruction instructions prntexpr  tinstr
%type <y\_nod> idlist l\_thing l\_list sortlist vsortlist funclist formlist termlist alg
%type <y\_nod> term func step att namedId d\_list
%type <y\_nod> formula cformula condition condfunc

%left La1
%left La2
%left La3
%left La4
%left La5
%right Ra
%right Ru
%left Func
%%
```

```
prog : line
| prog line

line : instructions '\n'

definition : FUNC namedId ':' vsortlist PFEIL Sort VORRANG att
| OTHER func ':' vsortlist PFEIL Sort att
| ATT Sort Id ',' Id ',' Id ',' Id
| BRA Id Id ':'  vsortlist PFEIL Sort att
| VAR idlist ':' Sort
| NUM Sort
| NUM idlist ':' Sort
| CST idlist ':' Sort
| SPEC Alg ':' alglist
| SPEC Alg ':' formlist
| ADDSPEC Alg ':' formlist
| SORT idlist

| ALG idlist
| TERM idlist ':' Sort

idlist : namedId
| namedId ',' idlist

l_thing : Const
| Term

l_list : l_thing
| l_thing ',' l_list

att  : /* NULL */
  | ATT namedId
| ATT '('
 d_list ')' instruction

namedId : Id
        | Const
        | Var
        | func
        | Sort
        | Alg
        | Term
```

```
VORRANG : /* nothing */
| LA1
| LA2
| LA3
| LA4
| LA5
| RA
| RU

condition : condfunc

condfunc : LB '(' termlist ')'
| IN '(' termlist ')'
| NIN '(' termlist ')'
| '(' term ')'

instructions  : instruction
| instruction ';' instructions

tinstructions : oinstructions
| oinstructions '\n' tinstructions

oinstructions : instructions

trenner : '\n'
| ';'

d_list : definition
 | definition ';' d_list

instruction : aninstruction
| definition
| '{'
tinstructions '}'
| WHILE condition instruction
| IF condition instruction
| IF condition instruction ELSE instruction
| FOR term EQ term TO term step DO instruction

aninstruction : /* Nichts */
| SYS String
| INC String
| REWR term ':' alglist
| prntlist
```

```
| PRTNUM String
| PRT prntlist
| OUT String
| DATA String
| DATA Const
| Term EQ term
| Term EQ REWR term ':' alglist
| Const EQ term
| EXE func Id
| LIFT '(' termlist ')' TO '(' termlist ')'
| DBG Const
| RETURN term
| QUIT

step  : /* NULL */
| STEP term

prntexpr : term
| String

prntlist : prntexpr
| prntexpr ',' prntlist

alglist : Alg2
| Alg2 ',' alglist

Alg2 : Alg
| '(' alglist ')'

sortlist : Sort
| Sort ',' sortlist

vsortlist : sortlist
| sortlist VARPAR

func : Func
| La1
| La2
| La3
| La4
| La5
| Ra
| Ru
| Bra
```

```
| Bra1

funclist : func
| func ',' funclist

term : Term
| LAST
| Const
| Var
| '(' term ')'
| term Ru
| Func term
| La1 term
| La2 term
| La3 term
| La4 term
| La5 term
| Bra termlist Bra
| Bra1 termlist Bra2
| Func '(' termlist ')'
| term La1 term
| term La2 term
| term La3 term
| term La4 term
| term La5 term
| term Ra  term

termlist : term
| term ',' termlist

formula : term EQ term

vinstr : /* */
| DO instruction

cformula : formula vinstr
          | IF condition formula vinstr

formlist : cformula
/* | cformula ';' formlist

%%
```